

LLD — Data Analyst / Behavioral Eng (Engagement Scoring)

Overview

This document is your Low-Level Design (LLD) for the **Data Analyst / Behavioral Eng** role (Python analytics scripts, trending metrics, emoji rating component, engagement scoring).

1) Goals

- Compute behavioral metrics from raw event data
- Produce trending metrics (top movers, slopes, deltas)
- Support emoji rating → numeric sentiment metrics
- Generate an engagement score (rule-based baseline + optional ML)
- Deliver runnable artifacts: `engagement_scorer.py`, `train.py`, `demo.ipynb`

2) Inputs & Outputs

Inputs (events)

Minimum columns:

- `user_id` (string)
- `event_type` (string)
- `ts` (datetime)

Recommended columns:

- `content_id`, `session_id`
- `dwel_time` (seconds), `scroll_depth` (0–1)
- `rating_emoji` (e.g., 😡 😐 😊 😍)

Outputs

- `features.parquet` : engineered features
- `scores.parquet` : engagement scores (0–100)
- `trend_report.json` : trending movers and summary
- `artifacts/model.*` : trained model (optional)

3) Architecture

Pipeline:

1. Ingest → 2. Clean/validate → 3. Feature engineering → 4. Score → 5. Trends → 6. Export

4) Data Design

Canonical event schema

- user_id, content_id, session_id
- event_type
- ts
- dwell_time, scroll_depth
- rating_emoji

Feature table (example: content-day)

Keys: `content_id`, `date`

- counts: views/clicks/shares/comments
- behavior: avg/p95 dwell, avg scroll
- emoji: count, avg, pos/neg rate
- score: engagement_score
- trend: delta_7d, slope_7d

5) Emoji rating component

Mapping (config-driven)

- 🤔=1, 😐=2, 😊=3, 😄=4, 😍=5

Aggregations

- emoji_avg_score
- emoji_pos_rate = score >= 4 / total
- emoji_neg_rate = score <= 2 / total
- apply minimum volume threshold; optionally smooth towards global mean

6) Engagement scoring

A) Rule-based baseline

Normalize components:

- $v = \log_{1p}(\text{views})$
- $c = \log_{1p}(\text{clicks})$
- $d = \text{clip}(\text{avg_dwell} / \text{dwell_ref}, 0, 1)$
- $s = \text{avg_scroll_depth}$
- $e = (\text{emoji_avg} - 1) / 4$

Score:

- $\text{raw} = w_v v + w_c c + w_d d + w_s s + w_e e$
- $\text{engagement_score} = 100 * \text{sigmoid}(\text{raw})$

Store explainability:

- views_part, dwell_part, emoji_part, etc.

B) ML-based (if labels exist)

- Train baseline model (LogReg) on engineered features
- Output probability → map to 0–100
- Persist: model + feature list + normalization stats

7) Trending metrics

For each entity (e.g., content_id):

- metric_7d, metric_prev7d
- delta, pct_change
- slope_7d (linear regression over daily series)

Output JSON:

- top_rising_by_pct, top_rising_by_slope, top_falling_by_pct

8) File-level design

`engagement_scorer.py`

Responsibilities:

- load events → clean → build features → score → compute trends → save outputs

Suggested CLI:

- --input --output_dir --granularity (user_day|content_day|session)
- --model_path (optional) --start_date --end_date

`train.py`

Responsibilities:

- build feature table → time-based split → train → evaluate → save artifacts

`demo.ipynb`

- reproduce feature build + scoring
- plots: score distribution, top entities, trend lines, emoji analysis

9) Testing checklist

- emoji mapping correctness (unknown emoji handling)
- aggregation sanity tests (counts, averages)

- score bounds test (0–100)
- leakage test (time split)

10) Open questions (fill these)

1. Score level: user / content / session / all
2. Labels available? yes/no (which label?)
3. Data source: CSV/Parquet/SQL/Notion export